

Lab 3 — Architectural Justification

Problem Statement #25 — Municipal Solid Waste Route Optimizer

Name: Priyanka B M **SRN:** PES1UG24AM207

Architecture Selection: We chose a Microservices Architecture for the Municipal Solid Waste Route Optimizer.

Why this architecture

Reason 1 — the workload isn't even across the system. Route Optimization Service is doing the heavy lifting here — recalculating routes for a fleet of 50 trucks across 10,000 bins, and it has to finish in under 45 seconds (that's our NFR-001). Meanwhile something like Complaint Management barely gets touched compared to that. If we'd gone with a single layered app, we'd end up scaling the whole thing just to keep the optimizer fast, which is wasteful. With microservices, we can throw more instances at just the Route Optimization Service during peak collection hours (early morning, basically) and leave everything else as-is.

Reason 2 — the components genuinely don't need to know about each other. Smart bin sensors pushing fill-level data, trucks getting routed, and citizens filing complaints about missed pickups are honestly three fairly separate concerns that just happen to share a database. Splitting them into Smart Bin Data Ingestion, Route Optimization, Fleet & Truck Management, and Complaint Management (each with its own provided/required interface, as shown in the component diagram) means a bug or a deploy in one doesn't force us to touch the others. That maps a lot more naturally onto microservices than trying to cram it into rigid presentation/business/data layers.

Security Advantage: Because each service only exposes what it needs to (Complaint Management only exposes the Complaint API, the Supervisor/Driver Portal only exposes the User Access API), a citizen hitting the complaints endpoint has no path into the truck-routing or fleet-management services. If one service's credentials or endpoint get compromised, the blast radius is limited to that service instead of the whole platform — which isn't something you get for free in a single monolithic app where everything shares the same process and trust boundary.

Performance Benefit: The 45-second recalculation requirement for 50 trucks / 10,000 bins is really only a problem for one component. Microservices let us scale just the Route Optimization Service horizontally (more instances, more compute) to hit that target during peak load, without over-provisioning the sensor ingestion pipeline or the complaint system, which doesn't need that kind of headroom. That keeps infra costs sane while still meeting the latency requirement where it actually matters.

Component Diagram

